

The Filter-Kruskal Minimum Spanning Tree Algorithm*

Vitaly Osipov, Peter Sanders, and Johannes Singler[†]

Abstract

We present *Filter-Kruskal* – a simple modification of Kruskal’s algorithm that avoids sorting edges that are “obviously” not in the MST. For arbitrary graphs with *random edge weights* *Filter-Kruskal* runs in time $\mathcal{O}(m + n \log n \log \frac{m}{n})$, i.e. in linear time for not too sparse graphs. Experiments indicate that the algorithm has very good practical performance over the entire range of edge densities. An equally simple parallelization seems to be the currently best practical algorithm on multicore machines.

1 Introduction

A minimum spanning tree (MST) of a graph $G = (V, E)$ is a minimum total weight subset of E that forms a spanning tree of G . The MST problem has been intensively studied in the past since it is a fundamental network design problem with many applications and because it allows for elegant and multifaceted polynomial-time algorithms. In practice (on sequential machines and in internal memory), two simple algorithms dating back at least half a century still perform best in most cases [17, 11, 6].

The *Jarník-Prim* algorithm [8, 20] grows a tree starting from an arbitrary node. Implemented using efficient priority queues, its running time is $\mathcal{O}(m + n \log n)$. Even with simpler priority queues, it performs well for random edge weights¹ – time $\mathcal{O}(m + n \log n \log \frac{m}{n})$ [18].

Kruskal’s algorithm [14] grows a forest in time $\mathcal{O}((m + n) \log m)$ by scanning the edges in order of increasing weight and adding those that join two trees in the current forest. In practice, *Kruskal* outperforms *Jarník-Prim* for sparse graphs. For denser graphs, *Kruskal* suffers from the $\mathcal{O}(m \log m)$ time needed for sorting all the edges. Therefore it is a natural idea to avoid sorting heavy edges that cannot contribute to the MST. Kershbaum and van Slyke [12, 17] do this by building a priority queue of edges in linear time. Then

Kruskal’s algorithm subsequently removes the lightest edge until $n - 1$ tree edges have been found. For random graphs with random edge weights, the MST edges are expected to be among the $\mathcal{O}(n \log n)$ lightest edges. Hence, we get an average execution time of $\mathcal{O}(m + n \log^2 n)$. Unfortunately, the stopping idea fails already if the MST contains a single heavy edge. Note that this can even happen for random edge weights: Consider a “lollipop graph” consisting of a random graph and an additional path of length k attached to one of its nodes. The MST needs all the path edges, about half of which will belong to the heavier half of the edges for random edge weights. Brennan [4, 19] implements the stopping idea more cache efficiently by integrating Kruskal’s algorithm with quicksort (*qKruskal*). Apply *Kruskal* to small inputs. Otherwise, as in quicksort, partition the edges into a light part and a heavy part. Recurse on the light part. If the MST is not complete yet, recurse on the heavy part.

A key idea for more robust improvements of *Kruskal* is *filtering* – early discarding of edges that connect nodes in the same component of the forest defined by the MST edges already found. In [12] filtering is applied to edges about to be sifted up in a heap based implementation of *Kruskal*. However, no analysis is given and heap based algorithms are unlikely to be efficient on modern machines due to the cache inefficiency. In this paper we investigate the idea to apply filtering to *qKruskal* – before recursing on the heavy part, remove all heavy edges that are within a component of the current forest. In Section 3 we explain the algorithm *Filter-Kruskal* in more detail. The analysis shows that for arbitrary graphs with random edge weights, *Filter-Kruskal* runs in expected time $\mathcal{O}(m + n \log n \log \frac{m}{n})$. Note that this is the same performance also achieved by *Jarník-Prim* using binary heaps [18]. The experiments reported in Section 4 confirm that *Filter-Kruskal* performs very well for both sparse and dense graphs. Moreover, *Filter-Kruskal* allows a more coarse-grained and hence more practical parallelization than *Jarník-Prim*.

More Related Work. MSTs can even be found in linear (expected) time [13, 9]. This algorithm can filter out edges without any sorting using sophisticated data structures that can check whether an edge e is the

*Partially supported by DFG grant SA 933/3-1.

[†]Universität Karlsruhe (TH), Germany,
{osipov,sanders,singler}@ira.uka.de.

¹here and in the following we use the following model for random edge weights: the edge weights are all different, can otherwise have arbitrary values, and are randomly permuted.

heaviest edge on the cycle defined by the minimum spanning forest (MSF) of an edge sample and e . However, such algorithms are complicated and large constant factors are involved. To check whether general edge filters are useful at all, [11] invests $\Theta(n \log n)$ preprocessing to allow for a better constant factor in filtering. This algorithm only significantly outperforms *Jarník-Prim* for rather dense graphs with weights that force m `decreaseKey` operations. Katajainen and Navalainen [10] refine the heap based algorithm from [12] by first performing a bucket sort of the edges. For uniformly distributed random edge weights, this yields (near) linear expected execution time. It should be noted that this result is quite different from ours: (1) bucket sorting does not work in the comparison based model, whereas our algorithms are comparison based. (2) the result in [10] only holds for “smooth” (i.e., close to uniform) distributions of independent random edge weights whereas our result holds for random permutations of arbitrary edge weights and in particular when edge weights are independently drawn from an arbitrary probability distribution. (3) for uniformly distributed edge weights, even basic *Kruskal* runs in (near) linear time if we use bucket sort. Indeed, experiments in [10] demonstrate that filtering does not help if bucket sorting can be used.

2 Preliminaries

Let $G = (V, E)$ denote an undirected, weighted graph with $|V| = \{1, \dots, n\}$. Let $m = |E|$. Since we need *Kruskal*’s algorithm as a subroutine, we outline it here for self-containedness. Figure 1 gives pseudocode that should be self-explaining. When *Kruskal* skips an edge $\{u, v\}$ that falls within a single component of T , this is safe because $\{u, v\}$ closes a cycle in T and is at least as heavy as all edges in T . In this situation, the *cycle* property of MSTs tells us that $\{u, v\}$ is not needed for an MST. The most sophisticated aspect of the algorithm is the Union-Find data structure P maintaining a partition of the nodes into components defined by the MST edges T found so far. P supports an operation `union` joining two partitions and an operation `find(v)` returning the node number of the *representative* of the partition of the node v . Indeed, the implementation will exploit that partitions are represented using *parent* references defining trees rooted at the representatives and that the paths leading to the roots are very short in an amortized sense (union-by-rank and path compression). In particular, if $m \gg n$, most path lengths will be one.

3 The Algorithm

Figure 1 gives pseudocode for *Filter-Kruskal*. Similar to [19], the basic approach is to use quicksort for sorting

Procedure

```
Kruskal( $E, T$  : Sequence of Edge,  $P$  : UnionFind)
  sort  $E$  by increasing edge weight
  foreach  $\{u, v\} \in E$  do
    if  $u, v$  are in different components of  $P$  then
      add edge  $\{u, v\}$  to  $T$ 
      join the partitions of  $u$  and  $v$  in  $P$ 
```

Procedure

```
filterKruskal( $E, T$  : Sequence of Edge,  $P$  : UnionFind)
  if  $m \leq \text{kruskalThreshold}(n, |E|, |T|)$  then
    Kruskal( $E, T, P$ )
  else
    pick a pivot  $p \in E$ 
     $E_{\leq} := \langle e \in E : e \leq p \rangle$ 
     $E_{>} := \langle e \in E : e > p \rangle$ 
    FilterKruskal( $E_{\leq}, T, P$ )
     $E_{>} := \text{filter}(E_{>}, P)$ 
    FilterKruskal( $E_{>}, T, P$ )
```

Function Filter(E, P : UnionFind)

```
return
 $\{\{u, v\} \in E : u, v \text{ are in different components of } P\}$ 
```

Figure 1: Pseudocode for *Kruskal* and *Filter-Kruskal*. T is a set of MST edges already known and P is the partition of V induced by T . When used as a standalone method, the procedures are called with empty T and a trivial partition P . The result is output in T .

the edges and to move the edge scanning part of *Kruskal* into the quicksort code. Hence, the algorithm now calls *Kruskal* on small² inputs and it calls itself for the lighter part of the edges. The only new ingredient at this level of abstraction is that before recursing on the heavier edges $E_{>}$, they are filtered. Filtering removes those edges that fall within the same component of the current node partitioning. Note that these edges are heavier than all edges in T and close a cycle in T . Hence, the cycle property implies that the filtered edges are not needed for an MST. The advantage of filtering is that filtered edges need not be sorted.

3.1 Analysis for Random Edge Weights. We will first show that we can essentially restrict the analysis to counting comparisons since this quantity is indicative of the total execution time:

LEMMA 3.1. *Let C denote the number of (edge weight)*

²As far as asymptotic performance is concerned, any choice of the function `kruskalThreshold` works as long as `kruskalThreshold( $n, |E|, |T|$ ) =  $\mathcal{O}(n)$` .

comparisons performed by *Filter-Kruskal*. Then *Filter-Kruskal* performs $\leq n - 1$ union operations, an expected number of $\leq 2m + C$ find operations, and $\mathcal{O}(m + C)$ work outside union and find operations.

Proof. Since *Filter-Kruskal* performs a union operation whenever it finds an MST edge, the claim on the number of union operations is obvious. Recall that *Kruskal* performs $2m$ find operation – one for each endpoint of an edge. In the worst case, *Filter-Kruskal* performs the same number of find operations in calls of *Kruskal* for subsets of edges. In addition, *Filter-Kruskal* performs some find operations in calls of *Filter*. More precisely, when a call of *Filter-Kruskal* picks a pivot of rank r , the corresponding call of *Filter* will perform $|E|$ comparisons and $2(|E| - r)$ find operations. The expected value of r is $|E|/2$ and hence, $\mathbb{E}[2(|E| - r)] = |E|$ – the same as the number of comparisons. All the remaining operations can be charged to comparisons or find operations proving the claim on the execution time.

LEMMA 3.2. ([21]) *Consider n union operations and M find operations on a union-find data structure with n elements using path compression and union by rank. Then the total execution time³ is $\mathcal{O}(M + n \log^* n)$ where $\log^* n$ denotes the iterated logarithm with $\log^* n = 0$ for $n \leq 1$ and $\log^* n = 1 + \log^* \log n$ otherwise.*

We now analyze *Filter-Kruskal* for random edge weights. Without loss of generality, we can assume that the edge weights are the set $1..m$.

THEOREM 3.1. *Given an arbitrary graph and random edge weights, the expected running time of *Filter-Kruskal* is $\mathcal{O}(m + n \log(n) \log \frac{m}{n})$.*

Proof. By Lemmas 3.1 and 3.2, it suffices to account for the number of edge weight comparisons. Suppose the MSF of the r lightest edges is known. This situation is analogous to the one in the linear time algorithm [9] when we filter against a random sample of r edges. Our approach is to integrate the backward analysis of the linear algorithm from [5] and a textbook analysis of quicksort [16].

Let edge i denote the edge with the i -th smallest weight. Edges i and j are compared at most once. Hence, we can count comparisons by looking at the indicator random variables X_{ij} , $i < j$, where $X_{ij} = 1$ if edges i and j are compared and $X_{ij} = 0$ otherwise.

³A more well known bound is $M\alpha(m, n)$ where α is the inverse Ackermann function. However, we need a bound that is linear in M .

Hence, the expected number of comparison is

$$\begin{aligned} \mathbb{E} \left[\sum_{i \leq m} \sum_{i < j \leq m} X_{ij} \right] &= \sum_{i \leq m} \sum_{i < j \leq m} \mathbb{E}[X_{ij}] \\ &= \sum_{i \leq m} \sum_{i < j \leq m} \mathbb{P}[X_{ij} = 1] . \end{aligned}$$

Edges i and j are compared only if one of them is picked as a pivot and neither of them is filtered out.

To bound the number of comparisons in the case of $i \leq n$ it already suffices to take only the first condition into account. Indeed, $\mathbb{P}[X_{ij} = 1] = 2/(j - i + 1)$ (see [16] for more details) and thus

$$\begin{aligned} \sum_{i \leq n} \sum_{i < j \leq m} \frac{2}{j - i + 1} &\leq 2n \sum_{j \leq m} \frac{1}{j} = 2nH_m \\ &\leq 2n(\ln m + 1) = \mathcal{O}(n \log n) \end{aligned}$$

where $H_m = \sum_{i \leq m} 1/i \leq \ln m + 1$ denotes the harmonic sum. We will need the last estimate several times – $\ln m \leq 2 \ln n = \mathcal{O}(\log n)$.

For the case $i > n$ we do take the second condition into account, but note that we filter out edge j only for the first pivot with rank $r \leq j$ that we encounter. We consider the following cases for the choice of the pivot. If $r > j$ then both i and j are passed to the same recursive subproblem and thus are not compared. Therefore, we are only interested in $r \leq j$. In case $i < r < j$ edge i and j are compared only with the pivot, but not with each other. If $r = i$ or $r = j$ then i and j are compared with probability $2/j$. For the case $r < i$ we distinguish two subcases, namely $r \leq n$ and $r > n$, that occur with probability n/j and $1/j$ for each choice of $n < r < i$ respectively. In the former subcase it is sufficient to assume that filtering is not effective and count only the probability of choosing i or j as a pivot, that is $2/(j - i)$. As for the latter subcase Chan [5] has proven that i (or, independently, j) will survive filtering with probability at most n/r . Hence, for the latter subcase we get comparison probability

$$\sum_{n < r < i} \frac{1}{j} \left(\frac{n}{r} \right)^2 \frac{2}{j - i + 1} \leq \frac{2n}{j(j - i + 1)}$$

where the latter estimate stems from the relation $\sum_{n < r < i} 1/r^2 \leq \sum_{r > n} 1/r^2 \leq 1/n$. Overall, we get

$$\begin{aligned} \mathbb{P}[X_{ij} = 1] &\leq \frac{2}{j} + \frac{n}{j} \cdot \frac{2}{j - i + 1} + \frac{2n}{j(j - i + 1)} \\ &= \frac{2}{j} + \frac{4n}{j(j - i + 1)} . \end{aligned}$$

For the $2/j$ term and the case $i > n$ we get the total contribution

$$\begin{aligned} \sum_{n < i \leq m} \sum_{i < j \leq m} \frac{2}{j} &= 2 \sum_{n < i \leq m} H_m - H_i \\ &\leq 2 \sum_{1 < i \leq m} H_m - H_i = 2(m-1). \end{aligned}$$

The latter sum can for example be evaluated with a computer algebra system like Maple or you write the sums into a triangular form such that each of $m-1$ columns sums to 2.

For the term $4n/j(j-i+1)$, we estimate

$$\begin{aligned} 4n \sum_{i=n+1}^m \sum_{j=i+1}^m \frac{1}{j(j-i+1)} &= 4n \sum_{j=n+2}^m \frac{1}{j} \sum_{\Delta=2}^{j-n+1} \frac{1}{\Delta} \\ &\leq 4n H_m \sum_{j=n+2}^m \frac{1}{j} = H_m(H_m - H_{n+2}) = \mathcal{O}\left(\log n \log \frac{m}{n}\right) \end{aligned}$$

We also give an informal argument why the complexity computed above is tight in the sense that using the sampling lemma from [5] we cannot expect a better result. Suppose, we had an algorithm that filters every edge with respect to all lighter edges “for free”. Then, $\sum_{n < i \leq m} n/i = \Theta(n \log \frac{m}{n})$ edges would survive this filtering (in expectation). Sorting those edges also yields the bound from Theorem 3.1.

Note that the term $n \log(n) \log \frac{m}{n}$ in the execution time of *Filter-Kruskal* can be simplified to $\mathcal{O}(n \log(n) \log \log n)$, i.e., for $m = \Omega(n \log(n) \log \log(n))$ we get linear execution time. This is up to a factor $\log n / \log \log n$ better than *qKruskal* for random graphs with random edge weights and recall that our result applies to *arbitrary* graphs with random edge weights.

Indeed, it seems that for random graphs with random edge weights we get even a better bound. Figure 2 indicates that the number of edge comparisons executed by *Filter-Kruskal* for graphs with $n \log n$ edges⁴ is proportional to $n \log n$ (at least the double-logarithmic upper bound from Theorem 3.1 is too pessimistic). This is quite strong evidence that the expected running time of *Filter-Kruskal* for random graphs with random edge weights is $\mathcal{O}(m + n \log(n))$: First observe that by Lemmas 3.1 and 3.2, the comparisons are representative of the asymptotic execution time. Second, for instances with *less* than $n \log n$ edges, the running time cannot be larger. Finally, random graph theory tells us that the $n \log n$ lightest edges will define the MST with high probability⁵. Hence, all the heavier edges will be filtered out anyway. We believe that a formal proof would not

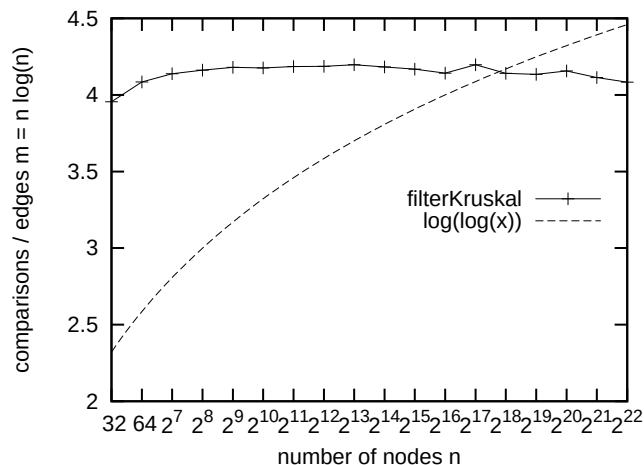


Figure 2: Number of edge comparisons performed by algorithm *Filter-Kruskal* for random graphs with $n \log n$ edges.

be too difficult by looking even closer at the structure of random graphs. In particular, since the number of nodes outside the giant component shrinks geometrically with the average degree, the probability that an edge survives filtering will also shrink geometrically with its rank divided by n . We believe that the same bound also applies to many other classes of graphs. Indeed we do not know a family of graphs for which *Filter-Kruskal* with random edge weights would take more than $\mathcal{O}(m + n \log(n))$ expected time.

3.2 Implementation. For $m \gg n \log(n) \log \log n$, most of the work is done in $\mathcal{O}(m)$ element comparisons performed using quicksort partitioning and the associated find operations in function *Filter*. Therefore, it makes sense to think about the constant factors involved here and to compare them with the constant factors involved in the *Jarník-Prim* algorithm. The number of comparisons (and associated finds) can be reduced by a constant factor by choosing pivots more carefully. Therefore, for an input segment of size k , our pivot is the median of a random sample of size $\sqrt{k}$. For the find operations, observe that most of the find operations will follow two single parent references to a common representative. This common case can be made fast as follows: When filtering an edge (u, v) , we first load the parent references pu and pv of u and v respectively. When $\text{pu} = \text{pv}$, we can immediately discard (u, v) . Otherwise, we complete the find operations for u and v and compare the results as usual.⁶ All in all, when most edges are

⁴Throughout this paper $\log x$ stands for $\log_2 x$.

⁵The threshold for connectivity is at $n \ln(n)/2$ edges.

⁶We use the same trick within Kruskal’s algorithm.

filtered out immediately in this way, the resulting $2m$ random memory references may dominate the running time for large, not too sparse graphs – the quicksort partitioning operations work cache efficiently.

Now let us compare this to the best case of the *Jarník-Prim* algorithm where for most edges (u, v) , we perform one random memory access to the distance value of v , compare it with the edge weight and discard (u, v) without accessing the priority queue. Since all edges are stored in both directions, we also get $2m$ random memory accesses. Hence, we can expect *Filter-Kruskal* and *Jarník-Prim* to perform similarly for large, sufficiently dense instances.

3.3 Parallelization. Most parts of *Filter-Kruskal* are well suited for parallelization – we can parallelize partitioning, sorting, and filtering. It is interesting to note that the find-operations done for filtering are logically completely independent although, due to path compression, there may be simultaneous read and write accesses to the same parent references. However, no matter how such operations are executed by the hardware, we will get correct results since we maintain the invariant that parent references eventually lead to the representative. Only the union-find operations in the *Kruskal*-call for the base case have to be executed sequentially. On average, these will only be called for a linear number of edges however.

We have implemented a multicore parallel version of *Filter-Kruskal*. Sorting and partitioning uses a parallel implementation of the C++ standard template library [22]. Partitioning is done using the inplace parallel algorithm from [23].

3.4 More Sophisticated Variants. Besides removing edges during filtering that are not needed for an MST, we can also identify some MST edges for good. Consider the multigraph G_s whose nodes are the components currently represented in the union-find data structure and whose edges are $E_s := \{(\text{find}(u), \text{find}(v)) : (u, v) \in E_>\}$ where $E_>$ contains the edges which survived filtering. For an edge $(u, v) \in E_>$, if $\text{find}(u)$ or $\text{find}(v)$ have degree one in G_s , then (u, v) is an MST edge. More generally, all edges outside the two-core⁷ of G_s correspond to MST edges and can be found in time linear in the size of G_s .

We have implemented the first variant of the algorithm (henceforth called *Filter-Kruskal+*) since degree-one nodes of G_s can be identified easily by maintaining a counter for each representative. Indeed, counter values

⁷The k -core of a graph G is the maximal vertex induced subgraph of G with minimal degree k . All k -cores can be found in linear time by subsequently removing small degree nodes.

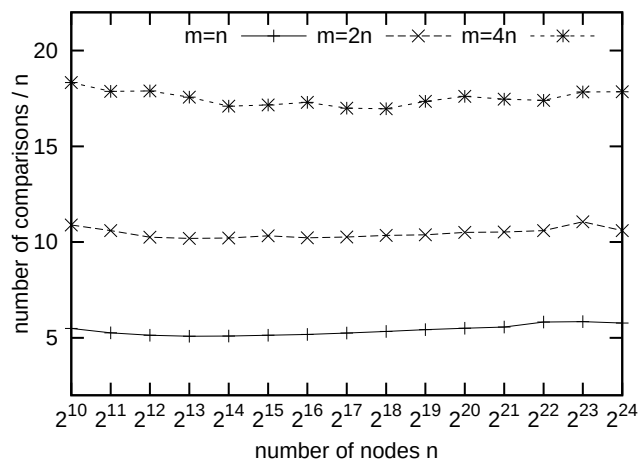


Figure 3: Number of edge comparisons performed by algorithm *Filter-Kruskal+* for random graphs.

0, 1, and “ > 1 ” suffice and can be stored together with the rank information in the unused parent references of component representatives (see also [6]). Figure 3 indicates that we get a near linear number of comparisons for sparse random graphs. Unfortunately, we will see that the overhead even for this simple measure is such that we see no improvement with respect to running time. Therefore we refrain from implementing the two-core refinement because this would even require building a proper adjacency-list representation of G_s which would probably be even slower. For the conversion time, refer to the Figure 11.

If we would take the time to build G_s explicitly, it would probably be even better to solve the MST problem recursively for G_s . With this measure we would move into the direction of a variant of the linear time randomized algorithm [9]. The difference would be that by taking advantage of random edge weights, we do not need a complicated data structure for filtering out heaviest edges on a cycle. Instead, we recurse on the lighter half of the edges and use a simple union-find data structure (which for this application could be made to run in linear time). The only missing ingredient to a linear time algorithm would be a node reduction algorithm. We could use the traditional Boruvka algorithm [3], or the sequential node reduction from [6]. The latter algorithm has an interesting deterministic variant where in each step, we remove the node with minimum degree. We have not implemented any of these algorithms because we do not think they could be competitive to our simple *Filter-Kruskal* algorithm in practice.

4 Experiments

Algorithms. We have experimented with *Kruskal*, *qKruskal* – the *Kruskal* modification from [19], *Filter-Kruskal*, *Filter-Kruskal+* from Section 3.4, and several variants of *Jarník-Prim* (*JP*). For *JP* we only show results for the best implementations: *pJP* for Irit Katriel’s implementation with pairing heaps [11] and *qJP*, our own implementation combined with Paredes’s quick-Heap priority queue [19]. *qJP* is considerably faster than Paredes’s own code since we use a faster graph representation (adjacency arrays rather than adjacency lists). We also use a multicore implementation of *Filter-Kruskal* (*Filter-Kruskal P* for P cores) and a version of *Kruskal* with parallel sorting of edges (*KruskalP*). Our graph data structure implements the interface of the Boost Graph Library [15], but uses a graph representation that is specific to the algorithm. For the variants of Kruskal’s algorithm this is simply an array of edges, for the *JP* algorithm, we use an adjacency array representation. We have also measured the time needed for converting between these representations.

Implementation. The implementation uses C++ with the GNU compiler version 4.3.1 and optimization level O3. The experiments were run on machine with two AMD Opteron 2.0 GHz quad-core CPUs.

Instances. Unfortunately, there is no established suite of real world instances for MST problems. Mostly, synthetic graphs families from the study [17] were used in the past. From these we use random graphs with random edge weights, random graphs with weights that force a *decreaseKey* for every edge, and random geometric graphs where n random points in the unit square are connected with their k closest neighbors with respect to Euclidean distance. Note that the resulting edge weights are not independent random numbers. We also use lollipop graphs with random edge weights where a path of length $n/2$ is appended to a random graph with $n/2$ nodes. Perhaps most interestingly, we have obtained a few instances generated by the image segmentation method by Jan Wassenberg (see also [7]), that was applied to satellite images, [1].

Random Edge Weights. Figure 4 shows the running time of the algorithms discussed above for random graphs with random edge weights. Lets first consider the middle graph with measurements for $n = 2^{16}$ nodes. *Kruskal*’s algorithm performs well for up to $8n$ edges where it is also well parallelizable. For more dense graphs, *JP* is better. None of the two priority queue variants is a clear winner. Quickheaps are a bit better for very sparse graphs whereas pairing heaps win for more dense graph. *qKruskal* does improve on *Kruskal* and outperforms *qJP*. *Filter-Kruskal* shows uniformly good performance over the entire range of

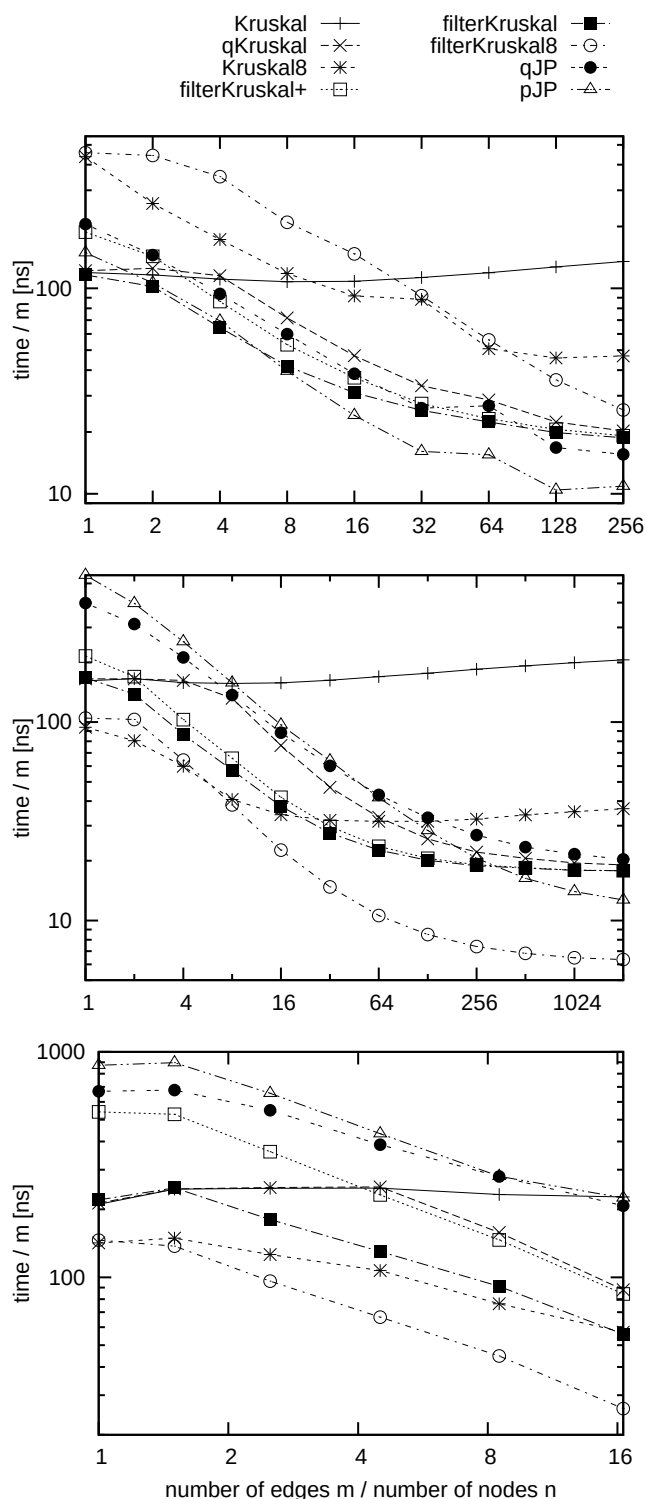


Figure 4: Time per edge for random graphs with random edge weights and 2^{10} (top), 2^{16} (middle), and 2^{22} (bottom) nodes.

densities. It is clearly better than *qKruskal* and only for rather dense graph it is still outperformed by *JP*. On 8 cores, *Filter-Kruskal* becomes the clear winner. Note that a parallel implementation of *JP* does not look promising except for very large, very dense graphs where parallelizing the innermost loop becomes interesting.

A more direct comparison to the sophisticated parallel MST implementations by Bader and Cong [2] would be interesting. However, they only report speedups for at least one million nodes and our codes are considerably faster than their codes if one simply scales the clock frequency of the machines. Hence, it currently looks like our algorithms are better at least for a small number of cores and in particular for small inputs.

Somewhat disappointingly, the “improved” Algorithm *Filter-Kruskal+* is always slightly slower than *Filter-Kruskal* – even the moderate additional effort for including degree-one edges never really pays off. We view this as an indication that even more complicated algorithms like [9] are even more far from being practical than we thought.

We have performed analogous experiments for smaller and larger random graphs with $n = 1024$ and $n = 2^{22}$ (see Figure 4). The ranking of the algorithms is similar as before, except that for very large graphs, *Filter-Kruskal* consistently outperforms *JP*. For small graphs, it is not astonishing that parallelization is not worthwhile. Here, *pJP* is the best algorithm throughout whereas *qJP* performs worse than both *pJP* and *Filter-Kruskal*.

Difficult Instances. For the difficult instances, we see in Figure 5 that *qJP* becomes extremely slow,

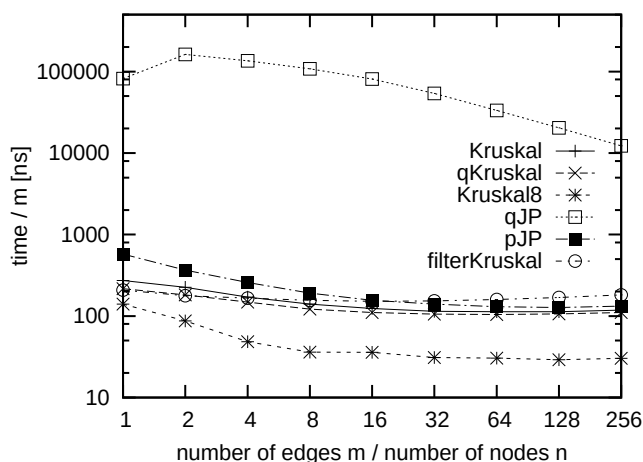


Figure 5: Time per edge for graphs that are bad for *JP* with 2^{16} nodes.

pJP and *Filter-Kruskal* are now somewhat worse than *Kruskal*, *qKruskal* yields a slight improvement over *Kruskal* and parallel *Kruskal* is the best algorithm.

Random Geometric Graphs. For random geometric graphs (see Figure 6) with 2^{16} nodes, we again have similar ranking as for random graphs, except that this time *Filter-Kruskal* outperforms the *JP* variants.

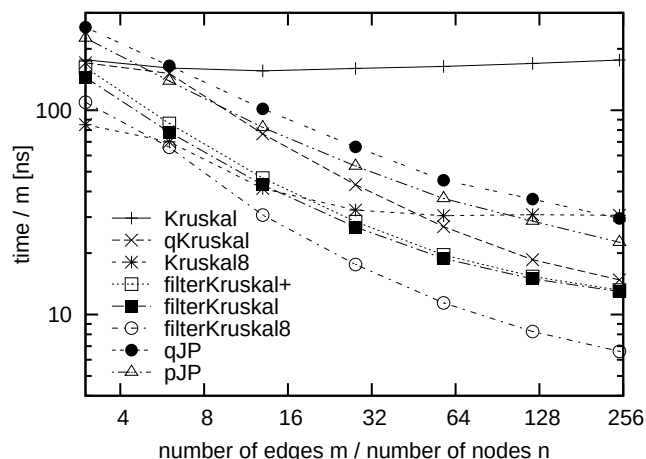


Figure 6: Time per edge for random geometric graphs with 2^{16} nodes.

Lollipop Graphs. For lollipop graphs (see Figures 7–9) we see similar result as for random graphs. The biggest difference is that *qKruskal* is now no better than *Kruskal*. *JP* outperforms sequential *Filter-Kruskal* for sufficiently dense graphs but not by a large margin.

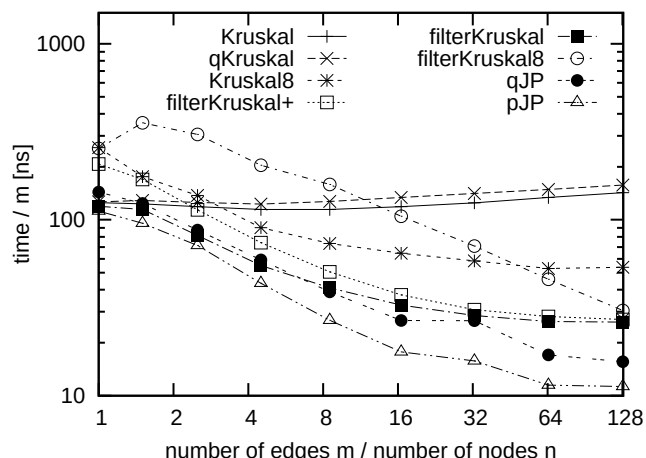


Figure 7: Time per edge for lollipop graphs with random edge weights and 1024 nodes.

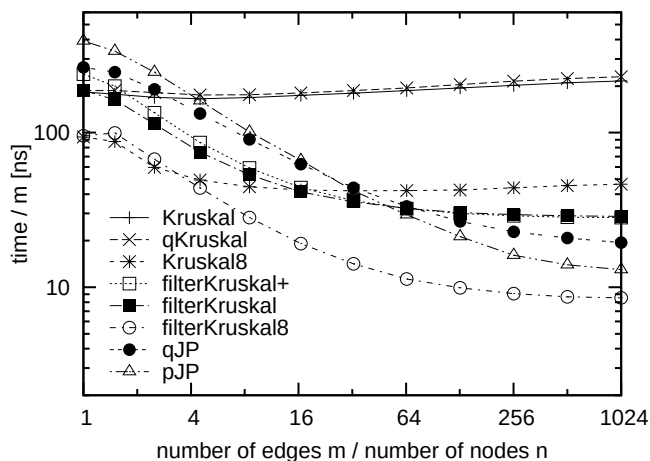


Figure 8: Time per edge for lollipop graphs with random edge weights and 2^{17} nodes.

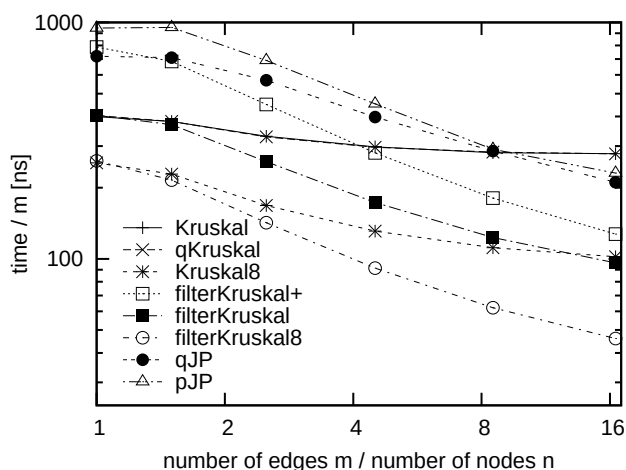


Figure 9: Time per edge for lollipop graphs with random edge weights and 2^{23} nodes.

Real World Instances. Surprisingly, for the image segmentation instances shown in Figure 10, *Filter-Kruskal* is again the best algorithm. As for lollipop graphs, *qKruskal* performs no better than *Kruskal* which is a confirmation of the intuition that this heuristics is not very robust.

Summary. The bottom line is that *Kruskal* remains a good algorithm for very sparse graphs and *Filter-Kruskal* and *pJP* contend for the best performance on more dense instances. We tend to give preference to *Filter-Kruskal* for three reasons. First, it shows good performance also for sparse graphs. Second, it is easily parallelizable, yielding a speedup of above two al-

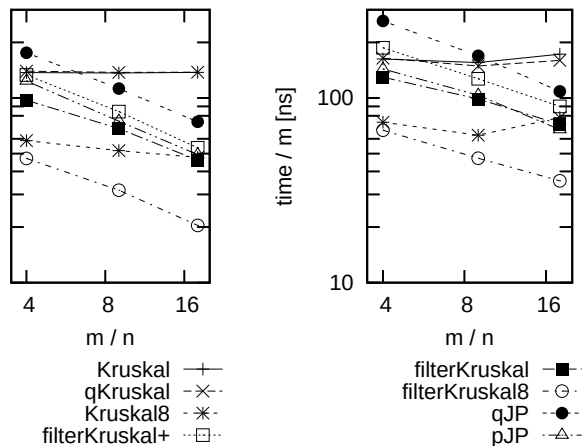


Figure 10: Time per edge for two families of graphs stemming from two satellite image segmentation problems for two different resolutions (800 000 nodes on the left, 4 163 616 nodes on the right) and different number of pixels considered in the “neighborhood” of each pixel.

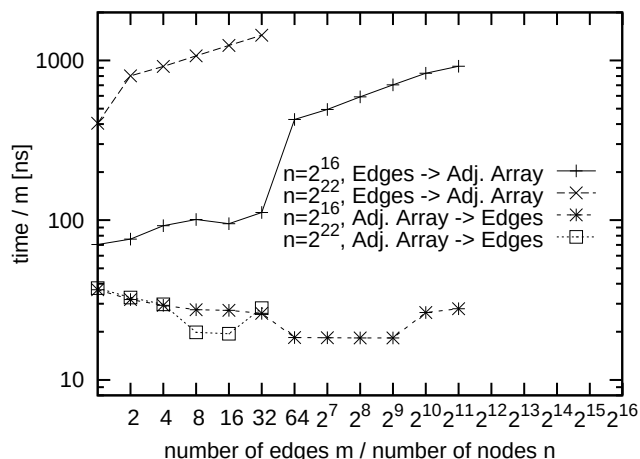


Figure 11: Time per edge for converting between edge sequences and adjacency arrays.

ready on low cost servers. Third, it only requires a list of edges as its input whereas *JP* needs a full fledged adjacency array. Figure 11 shows that building an adjacency array from a list of edges can take an order of magnitude longer than computing the MST!⁸ This

⁸We use the standard conversion algorithm that essentially performs a bucket sort on the endpoints of the edges [16, page 169], causing $\approx 10m$ cache faults. For large instances, this could be somewhat accelerated by using multipass algorithms but it is unlikely that the general picture gets reversed.

means that in cases where the adjacency array is not available, *Filter-Kruskal* will be much faster than *JP*. In contrast, building a list of edges from an adjacency array is very fast, indeed, the times given in Figure 11 are overestimates because we could fuse the loops for conversion and for the top level partitioning – scan the adjacency array and output to a partitioned array of edges.

5 Conclusions

Enhancing Kruskal’s algorithm with a simple filtering step leads to considerably improved performance. In particular, for arbitrary graphs with random edge weights, we obtain linear expected execution time for all but rather sparse graphs. It seems that this also applies to edge weights occurring in some real world applications. The resulting *Filter-Kruskal* algorithm not only outperforms Kruskal’s algorithm but is also competitive with the *Jarník-Prim* algorithm even for dense graph. *Filter-Kruskal* considerably outperforms *Jarník-Prim* if multiple cores are available or if the adjacency array is not given as part of the input.

The more sophisticated *Filter-Kruskal+* algorithm that also includes some edges into the MST without sorting, is interesting because it seems to yield a practical and relatively simple algorithm with average case linear execution time. Its somewhat disappointing practical performance might be offset in the future by more opportunities for parallelization. First experiments indicate that its nonparallelizable component, that is union-find operations within *Kruskal*-call for the base case, grows sublinearly with n (something like $n^{0.6}$).

Acknowledgements. We would like to thank Irit Katriel, Rodrigo Paredes, Dominik Schultes and Jan Wassenberg for providing graph generators, instances, and source codes. We also thank Martin Dietzfelbinger, Gonzalo Navarro, Rodrigo Paredes for fruitful discussions and Jyrki Katajainen for useful comments on the paper.

References

- [1] Space imaging gallery. <http://www.spaceimagingme.com/content/Gallery/>.
- [2] D.A. Bader and G. Cong. A fast, parallel spanning tree algorithm for symmetric multiprocessors (smmps). *Parallel and Distributed Computing*, 65(9):994–1006, 2005.
- [3] O. Boruvka. O jistém problému minimálním. *Práce, Moravské Přírodovědecké Společnosti*, pages 1–58, 1926.
- [4] J. J. Brennan. Minimal spanning trees and partial sorting. *Operations Research Letters*, 1(3):113 – 116, 1982.
- [5] T. M. Chan. Backwards analysis of the Karger–Klein–Tarjan algorithm for minimum spanning trees. *Information Processing Letters*, 67(6):303–304, 1998.
- [6] R. Dementiev, P. Sanders, D. Schultes, and J. Sibeyn. Engineering an external memory minimum spanning tree algorithm. In *IFIP TCS*, Toulouse, 2004.
- [7] P. F. Felzenszwalb and D. P. Huttenlocher. Efficient graph-based image segmentation. *International Journal of Computer Vision*, 59(2):167–181, 2004.
- [8] V. Jarník. O jistém problému minimálním. *Práce Moravské Přírodovědecké Společnosti*, 6:57–63, 1930. In Czech.
- [9] D. Karger, P. N. Klein, and R. E. Tarjan. A randomized linear-time algorithm for finding minimum spanning trees. *Journal of the ACM*, 42:321–329, 1995.
- [10] Jyrki Katajainen and Olli Nevalainen. An alternative for the implementation of kruskal’s minimal spanning tree algorithm. *Science of Computer Programming*, 3(2):205 – 216, 1983.
- [11] I. Katriel, P. Sanders, and J. L. Träff. A practical minimum spanning tree algorithm using the cycle property. In *11th European Symposium on Algorithms (ESA)*, volume 2832 of *LNCIS*, pages 679–690. Springer, 2003.
- [12] A. Kershenbaum and R. Van Slyke. Computing minimum spanning trees efficiently. In *ACM’72: Proceedings of the ACM annual conference*, pages 518–527, New York, NY, USA, 1972. ACM.
- [13] V. King. A simpler minimum spanning tree verification algorithm. *Algorithmica*, 18:263–270, 1997.
- [14] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7:48–50, 1956.
- [15] L. Q. Lee, A. Lumsdaine, and J. G. Siek. *The Boost Graph Library: User Guide and Reference Manual*. Addison-Wesley, 2002.
- [16] K. Mehlhorn and P. Sanders. *Algorithms and Data Structures — The Basic Toolbox*. Springer, 2008.
- [17] B. M. E. Moret and H. D. Shapiro. An empirical assessment of algorithms for constructing a minimum spanning tree. *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, 15:99–117, 1994.
- [18] K. Noshita. A theorem on the expected complexity of Dijkstra’s shortest path algorithm. *Journal of Algorithms*, 6:400–408, 1985.
- [19] R. Paredes and G. Navarro. Optimal incremental sorting. In *8th Workshop on Algorithm Engineering & Experiments*, pages 171–182. SIAM, 2006.
- [20] R. C. Prim. Shortest connection networks and some generalizations. *Bell Systems Technical Journal*, pages 1389–1401, November 1957.
- [21] R. Seidel and M. Sharir. Top-down analysis of path compression. *SIAM Journal of Computing*, 34(3):515–525, 2005.
- [22] J. Singler, P. Sanders, and F. Putze. MCSTL: The multi-core standard template library. In *13th Inter-*

national Euro-Par Conference, volume 4641 of *LNCS*, pages 682–694. Springer, 2007.

- [23] P. Tsigas and Y. Zhang. A simple, fast parallel implementation of quicksort and its performance evaluation on SUN enterprise 10000. In *PDP*, pages 372–381. IEEE Computer Society, 2003.